



Rubén Ruiz

Contents

Introduction	1598
The Importance of Scheduling	1599
Scheduling Is Hard	1600
The Relevance of Heuristic Scheduling	1600
Notation and Classification of Scheduling Problems	1601
Types of Scheduling Problems and Notation	1602
Scheduling Constraints	1603
Scheduling Objectives	1605
Basic Scheduling Heuristics	1607
Some Simple Dispatching Rules	1607
An Application to the Parallel Machine Problem	1608
Elaborated Dispatching Rules	1609
Advanced Heuristic Methods	1610
The NEH Heuristic for the Permutation Flow Shop and Makespan Criterion	1610
A Brief Introduction to the Shifting Bottleneck Heuristic for the Job Shop Problem	1612
Metaheuristics	1613
Main Concepts of Metaheuristics in Scheduling	1613
Iterated Greedy as an Example of a Simple- and High-Performing Method	1615
Conclusions	1617
Cross-References	1617
References	1617

R. Ruiz (✉)

Universitat Politècnica de València, València, Spain

AMAZON, Amazon.com, Inc., Seattle, WA, USA

e-mail: rruiz@eio.upv.es

Abstract

The scheduling of operations over resources is a relevant theoretical and practical problem with applications in many fields and disciplines, including the manufacturing industry. Scheduling problems are as varied as the reality they model. Additionally, some scheduling settings are among the hardest combinatorial problems there are. This is a perfect scenario for heuristic methods where high-quality robust solutions can be obtained in a short amount of time. This chapter concentrates on heuristics for production scheduling problems and summarizes the main results that range from simple rules to advanced metaheuristics. The importance of proper scheduling in practice is first highlighted, along with its difficulty and relevance. A summary of the scheduling notation is also given. Basic scheduling techniques, dispatching rules, combined rules, advanced heuristics, and an introduction to metaheuristics are also summarized in the chapter. While necessarily brief and incomplete, this chapter serves as an introductory point to those interested readers seeking to delve in the vast and rich world of scheduling heuristics. Some pointers to fruitful future research avenues are also provided. A large list of journal articles and monographs are provided as a reference for additional details and study.

Keywords

Scheduling · Dispatching rules · Heuristics · Local search · Metaheuristics

Introduction

In a wide sense, the term scheduling deals with the allocation of tasks to resources over a finite time horizon. These resources are usually limited and therefore, tasks have to share or, more precisely, compete for them. Depending on the type of scheduling problem, there might be interrelated decisions of allocation of tasks to resources, sequencing of the tasks assigned to each resource, assignment of start and finish times to each task, and so on. The objective is to find a schedule that optimizes a given criterion or criteria. As can easily be seen, this is a very broad definition. As detailed in [1, Chap. 1] and in [2, Chap. 1], scheduling is a decision-making process that is applied to a very wide range of situations in theory and in practice. As a result, resources can encompass from machines in a factory to tools or personnel in a plant or project. Resources might also model operation rooms in a hospital or cores in a computing cluster, etc. Similarly, tasks range from manufacturing operations to be performed in a factory to surgical interventions in an operation room. The applications are virtually limitless. During this chapter the focus is on one of the most studied applications of scheduling, which is manufacturing scheduling [3]. In manufacturing or shop scheduling, tasks commonly model client's orders or jobs, and resources are the machines or production lines needed to manufacture the products. While this field inside scheduling is arguably the most studied, other

monographs deal with other areas of scheduling in detail. Such is the case of Pinedo [2] for scheduling in services, Błażewicz et al. [4] for computer scheduling, Grosche [5] for scheduling problems in the airline industry, Randolph [6] for health-care scheduling, or Briskorn [7] for scheduling in sports. This short list is just an extract of the rich and varied literature on scheduling problems.

A salient characteristic of the manufacturing scheduling field is the sheer variety of real settings found in practice. It is easy to imagine that the scheduling problem found in a factory that produces cars is quite different from a factory that manufactures light bulbs to name just two random examples. Furthermore, the underlying scheduling problems are extraordinarily difficult from a mathematical perspective as most problems are of a combinatorial nature. This variety and the difficulty of the scheduling settings have resulted in a research field that is fragmented and overly specialized. More often than not, exact mathematical procedures are devised for specific scheduling settings where rigor and mathematical elegance are set as a goal in itself, possibly neglecting the general application or practical relevance of the proposed procedures. This has resulted in the well-known “research gap” between scheduling theory and practice, as recognized initially by Graves [8], Ledbetter and Cox [9], and Ford et al. [10]. All these studies pointed out the scant application of scheduling theory into practice. McKay et al. [11] and Olhager and Rapp [12] reached the same conclusions. Dudek et al. [13], MacCarthy and Liu [14], McKay and Wiers [15], McKay et al. [16], or McKay and Wiers [17] directly addressed, at different levels, this research gap. Different state-of-the-art reviews and studies about scheduling research like those of Reisman et al. [18], Ribas et al. [19], and Ruiz and Vázquez-Rodríguez [20] concluded that real scheduling problems are seldom studied. A more complete overview of these issues is detailed in Chap. 15 of [3].

The Importance of Scheduling

Without doubt, the aforementioned research gap is not to be attributed to a lack of interest in scheduling problems from industry. Good scheduling practice is of paramount importance in the manufacturing industry. Good schedules increase machine utilization, decrease manufacturing costs, and improve the fulfillment of delivery dates to customers ([1, 21, 22], among many others). Modern industries face a number of challenges that make scheduling as important as ever, to name just a few: (1) increasing product variety and/or configurations to reach a wider market, (2) reducing time to market, (3) shortening delivery dates to costumers, (4) high pressure in costs from emerging markets, (5) reducing production lot sizes, and (6) increasing frequency of orders from clients but of smaller size. All of the above are indicative of a transition from a mass manufacturing setting to a mass customization situation in a globalized economy. The optimized scheduling of hundreds, if not thousands, of production orders on a busy production shop floor is a must in order to meet these challenges.

Furthermore, good schedules help to identify production bottlenecks, streamline and stabilize production, minimize raw material shortages, and allow dealing with priorities in production orders. The conclusion is clear: nowadays factories must employ advanced scheduling tools in order to stay competitive amidst today's terribly competitive market.

Scheduling Is Hard

Sadly, scheduling models are remarkably hard to solve. In fact, most scheduling problems belong to the $\mathcal{NP}$ -Hard class of computational problems [23]. This is also true for most combinatorial optimization problems in many other fields inside operations research like location, routing, planning, etc. However, it can be argued that most real scheduling problems are particularly hard and in practice they seem to be more difficult to solve than problems in other areas. Take for example the well-known capacitated vehicle routing problem (CVRP) for which extensive literature exists. According to Laporte [24] and to the most powerful exact methods recently proposed for the problem [25], instances of up to 135 clients and 7 vehicles can be solved to optimality. In comparison, for the job shop scheduling problem (to be defined later), a wait of more than 20 years was needed to have small instances of 10 jobs and 10 machines solved to optimality by Brucker et al. [26], and no significant advances have been made for the optimal solution of instances with just 20 jobs. By all accounts, 20 jobs are way below the normal number of jobs being processed at any time in a real industry where hundreds of tasks are being processed on a daily basis.

Furthermore, real problems are much more complex than regular job shops and have many side constraints and generalizations. There is little hope of solving real problems to optimality. At this moment it is important to raise the question of whether optimal solutions are required in practice or not. Scheduling models are just that models of real problems that are, in the best case, good approximations of the modeled reality. Real data is often put into estimates that are fed to the models. In these circumstances it makes little sense to solve an approximated model with estimated data to optimality. While it is important to study theoretical problems and constructs and to propose exact approaches for such settings (as the insight gained from those studies is invaluable), real problems often require a different approach. This approach comes in the form of heuristics. Scheduling heuristics provide quick and fast approximated solutions to even the most intricate scheduling models. Moreover, and as it will be shown, such solutions are very close to optimality, at least in the cases where the optimality gap can be calculated or estimated.

The Relevance of Heuristic Scheduling

So far it has been stated that scheduling problems are as varied as the manufacturing reality they model and among the hardest combinatorial problems there are. This

has resulted in a research gap between the theory and practice of scheduling. Furthermore, there are many practical issues that arise when deploying scheduling systems in manufacturing firms. The study of such scheduling systems is not as rich as the theoretical works. Some references are Framinan and Ruiz, Framinan and Ruiz [27, 28], and Framinan et al. [3, Chaps. 13 and 14 and references therein]. To cope with the variety of real settings, general and robust heuristics are preferable to ad hoc exact approaches that are often tailored to specific problems and objectives. In particular, metaheuristics are powerful heuristic templates that, with little instantiation, are capable of solving a wide array of differing problems to almost optimality in reduced CPU times. Still, the development of such methods for practical applications is challenging and requires finesse as well as a deep knowledge of the main constructs and results. Such is the focus of this chapter. Examples from the most noteworthy basic heuristic procedures to the most advanced metaheuristics will be highlighted. Our natural inclination is toward general simple methods that include little to no problem-specific knowledge at all as they are the most suited to practical applications. More general approaches to heuristic and metaheuristic algorithms for scheduling problems are given by Morton and Pentico [29], Hoos and Stützle [30, Chap. 9], and Framinan et al. [3].

The rest of this chapter is organized as follows: section “[Notation and Classification of Scheduling Problems](#)” formalizes scheduling problems and provides a succinct explanation of the common notation used in the literature. Section “[Basic Scheduling Heuristics](#)” presents dispatching rules and simple heuristics for some classical and basic scheduling problems. Two advanced scheduling heuristics are presented in section “[Advanced Heuristic Methods](#).” A brief introduction to metaheuristics is provided in section “[Metaheuristics](#),” along with the description of a successful and simple methodology. Finally, section “[Conclusions](#)” concludes this chapter and some pointers for future research are given.

Notation and Classification of Scheduling Problems

Most production scheduling models define a set N of n jobs with consecutive indexes, i.e., $N = \{1, 2, \dots, n\}$. The resources are modeled in the form of a set M of m machines indexed as $M = \{1, 2, \dots, m\}$. All sets are assumed to be independent from each other. Subindexes j and k are used to refer to jobs and i and l to machines. Some basic information associated with jobs and machines is as follows:

- A processing route. For a given job j , the route is an ordered list of machines that have to be visited in order for the job to be completed at the factory. If a job has to visit three machines (e.g., 2, 1, and 6), it is said that it has three tasks. Usually, and following this small example, the task in machine 1 cannot start until the previous task of the same job in machine 2 has finished.
- Processing times. These are commonly denoted as p_j or p_{ij} . This is the amount of time that a given job j needs at machine i . This time is fixed and usually it cannot

be interrupted. Machine i is busy while processing job j and cannot be occupied during this time by any other job.

- Due dates and weights. These are denoted by d_j and w_j , respectively. The due date models the time at which job j must be completed in the factory to be delivered to the customer. The weight captures the relative importance of the job. This data will be used later for the objectives to optimize.

Of course, a wealth of additional data might be needed for the jobs and machines, depending on the real problem to solve. However, and for the sake of brevity, the previous data suffices as a bare minimum. Furthermore, in the course of this chapter, it is assumed that all data is known and deterministic. Most figures are usually represented by nonnegative integers for simplicity. The literature on stochastic scheduling is vast. The interested reader is referred to two recent monographs and the references therein for more details [31, 32]. Additionally, the part on stochastic models in Pinedo [1] is an excellent starting point.

Types of Scheduling Problems and Notation

The literature on production scheduling is mostly partitioned according to how the machines are laid out on the production floor and also on the characteristics of the processing routes of jobs. With this in mind, the main scheduling problems relate to the following scenarios:

- Single machine problems. This is the simplest setting. There is a known number n of jobs, and each one has to be processed on a single machine, following a sequence. Most of the time only a vector p_j with the processing times of the jobs on this single machine is needed.
- Parallel machine problems. Picture the previous case but with m machines disposed in parallel. Jobs still have to be processed by one single machine, but in this case it has to be one out of the m available. Therefore, an assignment problem precedes that of the sequencing problem among all jobs assigned to each machine. The m parallel machines might be identical, uniform, or unrelated. In the case of identical parallel machines, the processing times for the jobs are independent of the machines. In the unrelated case, the processing time of a job j might change depending on which machine it is assigned to. Therefore, a processing time matrix p_{ij} is needed. The uniform parallel machine case is an intermediate situation where machines are faster or slower for all jobs by a factor.
- Flow shops. Here the m machines are disposed in series and all jobs must visit all machines in the same order. This means that the processing route is identical for all jobs and is usually assumed to be $1 \dots, m$. Jobs are broken down into m tasks with known processing times p_{ij} . As stated before, any task for a given job is unable to start before the preceding task at the previous machine has finished. A flow shop is a very common production machine on production or assembly lines.

- Job shops. Here the main difference from flow shops is that each job has a potentially different processing route. Therefore not all jobs must visit all machines and nor do they necessarily have to be in the same order.
- Open shops. This is a much less common setting in which each job has to visit a given number of the m machines, but the route is open for the scheduling method to determine.
- Hybrid shop environments. Most real problems are actually a combination of the previous shop settings and parallel machines. Instead of having m machines disposed in series, there are m stages where at each stage i , there might be a single machine or m_i parallel machines, be these identical, uniform, or unrelated. This is, by far, the most complex and, at the same time, most general production setting.

Note that the previous short list is a massive simplification. Other production settings include assembly shops, manufacturing cells, flexible manufacturing systems (FMS), batch machines, and so on. In any case, the previous classification is the most employed one. Following this classification, the most common notation was proposed by Rinnooy Kan [33] and is a triplet $\alpha/\beta/\gamma$. The first field α covers the previous scheduling types with the values 1, P, Q, R, F, J, and O for single machine, identical parallel machines, uniform parallel machines, unrelated parallel machines, flow shop, job shop, and open shop problems, respectively. For more advanced layouts, like hybrid shops, there are extensions of this notation, like the one provided by Vignier et al. [34] and reviewed in Ruiz and Vázquez-Rodríguez [20]. The second field β is concerned with the constraints present in the problem, and the last field γ captures the objective function to optimize. These two last fields are succinctly summarized in the following sections.

Scheduling Constraints

As mentioned, there are as many scheduling constraints as product varieties and types of factories. Following Framinan et al. [3], the different scheduling constraints can be classified if they affect the processing of the jobs, operations, transportation, storage, or others:

- Process constraints. In this broad category, it might be mentioned all situations that affect the flow of the operations in the factory. Here one can find job/task precedence constraints ($\beta = \{\text{chains}, \text{intree}, \text{outtree}, \text{tree}\}$ depending on whether the precedence constraints form chains, convergent trees, divergent trees, or general trees, respectively). Here job/tasks might be subject to precedences, i.e., some jobs might not start before others have finished. Another frequent processing constraint is the setup times. Setup times model all nonproductive operations that have to be performed at machines after finishing jobs and before starting new ones. Cleaning, fixing, and reconfiguration are common setup operations. Setup times might be anticipatory/non-anticipatory and/or sequence

independent/dependent. As such, S_{ijk} models the amount of time that it is needed to setup machine i after processing job j and before processing job k . The literature on scheduling with all sorts and types of setup times is incredibly large. In parallel and hybrid layouts, jobs might have machine eligibility constraints, indicating that not all available machines can process certain jobs. Machines might also be subject to unavailability periods and/or breakdowns. Jobs might also have more general processing routes requiring stage/machine skipping or recirculation (revisiting one or more stages/machines more than once). This last circumstance is indicated as $\beta = \text{recrc}$. There are many more process constraints than the ones mentioned here. Consult Pinedo [1] or Framinan et al. [3, Chap. 4] for a more detailed account.

- Operations constraints. On many occasions the operation or task of a job on a given machine is more complicated. For example, tasks might be interrupted arbitrarily to be repeated or resumed later, possibly after having processed a task from a different job in between. Tasks from the same job might also be split to be processed by different parallel machines. Tasks/jobs might be subject to release dates, i.e., $\beta = r_j$ indicates that job j might not start processing at its first machine before time r_j due to, for example, unavailability of raw materials. Apart from release dates, there might be due dates, deadlines and due windows, etc. Another frequent characteristic is that jobs might not be allowed to wait in between machines ($\beta = \text{nwt}$). This is common in the processing of prepared foods in order to avoid breaking the cold chain. At the same time, there might be forced minimum waits for a job in between stages (cooling down, drying off), maximum waiting times, waiting windows, or even overlaps between consecutive tasks of a job. Similarly, processing times might be subject to all types of additional situations, like learning, deterioration or position dependency, etc.
- Transportation constraints. It is very common in real industries to have means of transporting unfinished jobs between machines. The list here is endless as one might have from simple manually operated pallet jacks for moving products from machine to machine to forklift trucks or even complex automated guided autonomous vehicles. Conveyor belts, handling robots, and other equipment are very common too. Depending on the scenario, transportation might be modeled as a simple fixed transportation time in between machines to very complex scheduling-routing problems when the transportation is an essential part of the process and transportation equipment is limited and expensive.
- Storage constraints. Similar to transportation constraints, production in course has to be stored somewhere. Again, and depending on the real situation, storage might be of no concern as space might be ample and/or the type of product being produced small and easy to store. However, in other situations the handling of unfinished products might be so challenging as to be a central constraint in the factory. Imagine, for example, the production of commercial airplanes where moving an airplane frame from one processing station to another is far from a trivial matter.

- Other constraints. Apart from machines one might have additional resources, like special tools, fixtures, or even the personnel operating machines. This personnel might have different degrees of skill. In some factories, machines might be able to perform more than one operation in parallel. Manufacturing firms might have more than one production center. Processing and/or setup times might be controllable depending on the resources (i.e., personnel) dedicated to them. There might be timing/shift constraints, safety concerns, etc. As indicated, the list of potential situations in practice is very long.

Scheduling Objectives

Finally, field γ is concerned with the scheduling objectives to optimize. In order to understand these, there is a need for some additional notation:

- C_j is the completion time of job j in the shop. It models the time at which the last task is completed at the last machine.
- L_j is a measure of the lateness of job j with respect to its due date d_j . $L_j = C_j - d_j$.
- T_j is a measure of the tardiness of job j where $T_j = \max \{L_j, 0\}$.

This is just a small extract of the many possibilities. Similarly, earliness can be defined when jobs finish before their due dates. From these definitions the two main groups of objective functions are those related with completion times and those with due dates:

- Completion time based. The most common one is the minimization of the maximum completion time among the jobs, usually referred to as makespan or $C_{\max} = \max \{C_1, C_2, \dots, C_n\}$. Makespan is mainly related to maximizing machine utilization. The second most studied objective is the minimization of the total flow time, defined as $\sum C = \sum_{j=1}^n C_j$.
- Due date based. The most common is the minimization of the total tardiness of the jobs $\sum T = \sum_{j=1}^n T_j$. However, the weighted version or $\sum wT = \sum_{j=1}^n w_j T_j$ is more realistic as being tardy for some important jobs should be considered over being tardy for irrelevant orders.

There are tens, if not hundreds, of potential objectives. A much more detailed list is given in Chap. 5 of Framinan et al. [3]. Furthermore, the reality is often multiobjective. The extension of the γ notation for multiple objectives in scheduling is detailed in T'Kindt and Billaut [35].

Let us now give a simple example of the $F/prmu/C_{\max}$ problem or the permutation flow shop problem with makespan criterion. The example with the total flow time objective or $F/prmu/\sum C$ will be used. There are five jobs and four machines. The processing times are shown in Table 1.

Table 1 Processing times (p_{ij}) of a flow shop problem with five jobs and four machines

Machine (i)	Job (j)				
	1	2	3	4	5
1	29	37	95	85	65
2	30	62	59	11	55
3	27	21	70	62	67
4	2	6	82	80	57

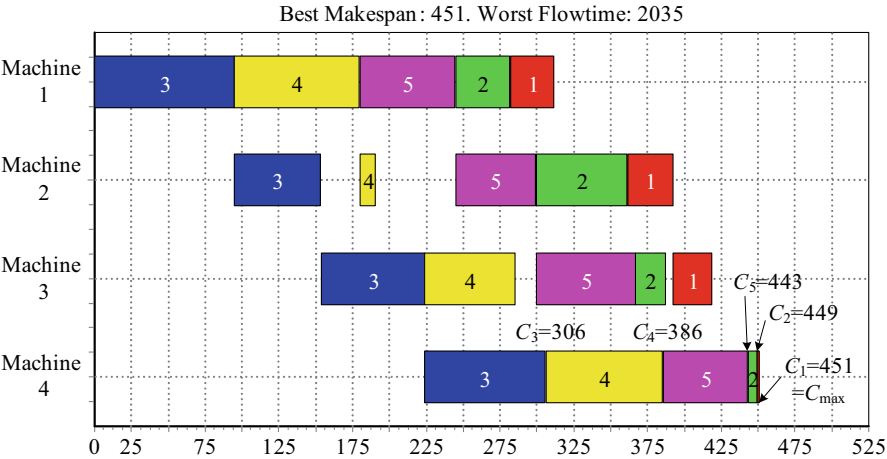


Fig. 1 Gantt chart for the permutation flow shop example with the best $C_{\max}$ which corresponds also with the worst possible $\sum C$ value

The permutation flow shop is a problem of determining a sequence for the jobs so there are $5! = 120$ possible schedules in this small example. If one calculates all of them, the best makespan value we obtain is $C_{\max}^* = 451$ with the permutation or job sequence $\pi_{\max}^* = \{3, 4, 5, 2, 1\}$. The worst makespan value is $C_{\max}^w = 522$ (15.74% worse) resulting from the permutation $\pi_{\max}^w = \{1, 2, 4, 5, 3\}$. What is interesting about this example is that the best flow time value is $\sum C^* = 1,464$, resulting from the permutation $\pi_{\sum C}^* = \{1, 2, 4, 5, 3\}$. As one can see, $\pi_{\max}^w = \pi_{\sum C}^*$, i.e., the worst makespan solution is equal to the best flow time solution. Furthermore, the contrary is also true. For this example, the worst flow time value is $\sum C^w = 2,035$ (38.93% worse than the best), resulting from the permutation $\pi_{\sum C}^w = \{3, 4, 5, 2, 1\}$. As one can see, $\pi_{\max}^* = \pi_{\sum C}^w$. The conclusion is that the best possible makespan value can result in the worst possible flow time and vice versa. The two permutations in this example are depicted in Figs. 1 and 2, respectively. The completion times of all jobs in the last machine have been indicated for further examination. While this is a cherry-picked example, it is easy to find examples such as this for almost any number of jobs and machines.

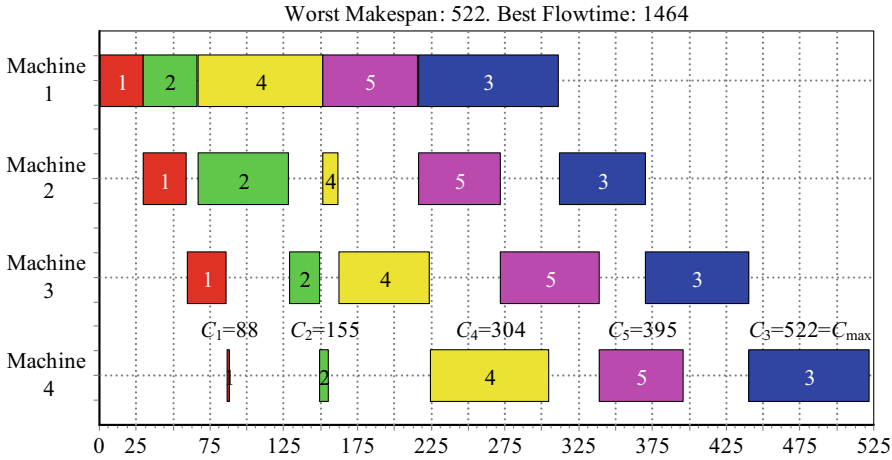


Fig. 2 Gantt chart for the permutation flow shop example with the best $\sum C$ which corresponds also with the worst possible $C_{\max}$ value

Basic Scheduling Heuristics

In the early twentieth century, modern production plants started using scheduling techniques to increase productivity. Human schedulers planned production manually without any aid other than pen and paper. The situation today has not changed much for most industries where basically spreadsheets are used to program production, but simple rules are used to obtain effective schedules. These simple rules are often referred to as list scheduling rules, dispatching rules, or priority rules. Some of them will be briefly explained. More advanced dispatching rules will be explained later.

Some Simple Dispatching Rules

Dispatching rules are usually forward heuristics where new tasks or jobs are sequenced as they arrive or as their processing becomes feasible (i.e., all preceding tasks have been already scheduled). A decision is made over a set of eligible tasks, referred to as φ . Some simple dispatching rules are the following:

- First Come First Served (FCFS): Here the job or task that entered the list of eligible tasks first is scheduled, i.e., the eligible tasks are sequenced in a first in first out (FIFO) order.
- Shortest Processing Time (SPT) First: The pending eligible with the shortest processing time goes first, i.e., given m machines, the task k that satisfies the following expression is chosen: $p_{ik} = \min_{j=1}^{|\varphi|} \{p_{ij}\}$. While simplistic, SPT

results in the optimal solution for the single machine total flow time problem ($1 \parallel \sum C$). The contrary rule to SPT is the longest processing time (LPT) first rule.

- Earliest Due Date (EDD) First: This is a dispatching rule that considers due dates. Basically the pending task with the earliest due date is scheduled first. EDD is optimal for the problems $1 \parallel T_{\max}$ and $1 \parallel L_{\max}$.

Note that these dispatching rules are extremely fast; all of them have a computational complexity of $\mathcal{O}(n \log n)$, n being the number of jobs or tasks to be scheduled. They have many other advantages like their simplicity and ease of application. Furthermore, they are easy to understand which is something valued by human schedulers. There are literally hundreds of dispatching rules and they have been comprehensively studied. It is suffice to cite just some of the exiting reviews in Panwalkar and Iskander, Blackstone et al., Haupt, and Jayamohan and Rajendran [36–39].

An Application to the Parallel Machine Problem

In general, dispatching rules, albeit general, fast, and simple, are overly myopic and produce subpar results, especially when compared to more elaborated constructs. However, in some cases they provide strong solutions. The following example applies to the identical parallel machine problem with makespan criterion or $P \parallel C_{\max}$. In this problem only the assignment of jobs to machines is relevant, as the sequence at each machine has no effect on the maximum completion time. There are n jobs to be scheduled in m identical parallel machines with the only input data being a vector with the processing times p_j . The $P \parallel C_{\max}$ is $\mathcal{NP}$ -Hard even for two machines [40].

Real results from an exam scheduling problem of a big company in the USA (Pearson VUE) are presented. This company has 5000 test centers in 175 countries in which all types of certifications, admissions, and exams are taken. The problem solved is the scheduling of different exams every week of a given year at each center in the USA so as to minimize the number of hours that each center is open to the public. Each center has a number of seats in which exams can be taken. Therefore, each center is modeled as an identical parallel machine problem in which the number of seats is the number of machines. The objective is to minimize the makespan as this translates directly into a minimization of the opening hours. The different exams are job types, and registrations for a given exam by users at a given week are the jobs to be scheduled at the given center in that week. All 219 centers in the USA for the 52 weeks of 2012 were solved. This resulted in a total of 11,261 different identical parallel machine problems with 1,244,063 jobs in total. The largest center had 308 machines (seats) available, and the biggest number of jobs to solve in a given week and center was 638.

The algorithm of choice is the aforementioned LPT, which for the parallel machine case is simple. Jobs are ordered from longest to shortest processing times, and the first m are assigned to the m parallel machines. Then, each job in the ordered list is assigned to the first freed machine. This LPT is known to have a worst-case performance guarantee of $\frac{4}{3} - \frac{1}{3m}$ [41]. This means that for large m values, the solution given by LPT should be at most around 33% worse than the optimal makespan. Now the results of the LPT with the solutions obtained by a state-of-the-art solver (CPLEX 12.5) are compared. The experiments were run on a computing cluster with 240 cores in total (XEON E5420 processors running at 2.5 GHz.) and 480 GBytes of RAM memory. Around 93% of problems could be solved to optimality and the average gap obtained was 0.57%. However, more than 80 h of CPU time were needed in the cluster (time limit for each problem solved was set to 300 s, which was reached in 851 cases). Comparatively speaking, the LPT resulted in a gap in respect to the MIP optimal solution or lower bound of just 0.91%. The total CPU time of the application of the LPT for all 11,261 problems was just 17.16 s on a single computer and most of this time was the processing of the input data and generation of the solution files.

The conclusion is clear. For some problems, including real ones, the application of simple dispatching rules can result in extraordinary performance in negligible CPU times.

Elaborated Dispatching Rules

Many authors have explored more intricate procedures. A clear example is the apparent tardiness cost (ATC) composite dispatching rule proposed by Vepsäläinen and Morton [42]. This rule is excellently explained in Pinedo [1]. The ATC rule was presented for job shop problems with total weighted tardiness criterion but has been successfully applied to different problems as well. The rule is applied to all jobs and an index is calculated. The job with the highest index is scheduled and then the rule is calculated again for the remaining $n - 1$ jobs. In total ATC is computed $n \cdot (n + 1)/2 - 1$ times resulting in a computational complexity of $\mathcal{O}(n^2)$. The index is calculated for each job j , and each time a machine is free as

$$I_j(t) = \frac{w_j}{p_j} \cdot e^{\left(\frac{\max\{d_j - p_j - t, 0\}}{K \bar{p}} \right)}$$
 where $\bar{p}$ is the average of the processing times and K is a parameter that needs to be fixed. ATC was expanded for the consideration of setup times by Lee et al. [43] resulting in the apparent tardiness cost with setups or ATCS.

Here the index is further complicated: $I_k(t, j) = \frac{w_k}{p_k} \cdot e^{\left(\frac{\max\{d_k - p_k - t, 0\}}{K_1 \bar{p}} \right)} \cdot e^{\left(-\frac{s_{jk}}{K_2 \bar{s}} \right)}$. As one can easily see, the calculations quickly become cumbersome; the rule is also harder to understand and to apply which basically defeats many of the advantages of dispatching rules. The benefits are better performance and objective function values.

Advanced Heuristic Methods

Advanced heuristics usually include problem-specific knowledge exploiting theoretical properties of scheduling problems. Alternatively, some methods aim at deriving better schedules starting from previous schedules. These latter approaches have been known as improvement heuristics. The distinction between basic or constructive heuristics and improvement heuristics is not so clear on many occasions. In any case, most of the time constructive heuristics have a natural stopping criterion, i.e., the method stops when a complete schedule or sequence has been built. Improvement heuristics don't have such a direct stopping criteria.

Most improvement heuristics and metaheuristics (to be succinctly reviewed in the next section) are strongly based on local search techniques. Local search is a heuristic exploration technique with a strong methodological background ([30], among many others). Local search requires at least the following ingredients: (1) A definition of what is a neighboring solution of a sequence; (2) a way to explore the neighbors of a solution; (3) an acceptance criterion, i.e., when a neighboring solution must replace the current one; and (4) when and how to stop the local search. Usually the main and most important issue is how to represent a solution and how to define a neighborhood from that representation. In most scheduling problems, solutions are indirectly represented as permutations of integers. The most employed neighborhoods are swap and insert where two jobs exchange their positions in the sequence in the case of swap and a job is extracted from its position and inserted into another position in the case of insert. In the following section, a very successful heuristic for the flow shop problem is detailed. It is based on the exploration of the insertion neighborhood in a constructive phase. Later another effective heuristic is briefly described but this time for the job shop problem and based on a different methodology the decomposition of the scheduling problem into smaller subproblems.

The NEH Heuristic for the Permutation Flow Shop and Makespan Criterion

The NEH heuristic of Nawaz et al. [44] is one of the most successful advanced heuristics. It was originally proposed for the permutation flow shop with makespan criterion ($F/prmul/C_{\max}$). It is powerful and capable, with hundreds of applications and references to many scheduling problems. Reviews like those of Turner and Booth [45], Taillard [46] or Ruiz and Maroto [47] recognize its capacity. The NEH exploits the fact that jobs with large total processing times must be sequenced as early as possible. The NEH consists of three steps:

1. The total processing times of the jobs in all machines are calculated: $P_j = \sum_{i=1}^m p_{ij}$, $j = (1, \dots, n)$.
2. Jobs are sorted in descending order of P_j , so that the first job is the one that has the highest time requirements. These sorted jobs are stored in a list ℓ .

3. The first two jobs of the list ℓ , referred to as $\ell_{(1)}$ and $\ell_{(2)}$, are extracted. The two possible partial sequences made up from these two jobs are constructed and their respective makespan values calculated. In other words, the two following partial sequences are tested: $\{\ell_{(1)}, \ell_{(2)}\}$ and $\{\ell_{(2)}, \ell_{(1)}\}$. From the two sequences, the one with the minimum partial $C_{\max}$ value is retained for the next iterations.
4. The main loop of the NEH starts here. At each iteration, the job at position k of the list ℓ , i.e., $\ell_{(k)}$, $k = (3, \dots, n)$ is extracted. This job $\ell_{(k)}$ is inserted into all possible solutions of the partial incumbent sequence, and the sequence resulting in the lowest partial $C_{\max}$ value is retained for the next iteration. For example, let us suppose that the best partial sequence obtained up to a given point is $\pi_p = \{\ell_{(1)}, \ell_{(3)}, \ell_{(2)}\}$ and that $k = 4$; therefore, job $\ell_{(4)}$ has to be inserted in the four possible positions of the partial sequence: $\pi_{p+\ell_{(4)}}^1 = \{\ell_{(4)}, \ell_{(1)}, \ell_{(3)}, \ell_{(2)}\}$, $\pi_{p+\ell_{(4)}}^2 = \{\ell_{(1)}, \ell_{(4)}, \ell_{(3)}, \ell_{(2)}\}$, $\pi_{p+\ell_{(4)}}^3 = \{\ell_{(1)}, \ell_{(3)}, \ell_{(4)}, \ell_{(2)}\}$, and $\pi_{p+\ell_{(4)}}^4 = \{\ell_{(1)}, \ell_{(3)}, \ell_{(2)}, \ell_{(4)}\}$. For the next step ($k = 5$), the sequence with the lowest partial $C_{\max}$ value from the four previous possible sequences is retained.

As one can see, NEH is basically based upon insertions of jobs into a partial sequence that is constructed step after step. Step 1 has a complexity of $\mathcal{O}(nm)$. The second step is just an ordering and takes $\mathcal{O}(n \log n)$. The third step requires just two calculations of two small sequences and can be disregarded. Most CPU time is needed by step 4. In step 4 there is a loop of $n - 2$ iterations where at each step k , k insertions of job $\ell_{(k)}$ are carried out into a partial sequence that contains $k - 1$ jobs, and for each insertion one needs to calculate the $C_{\max}$ value of k jobs (including the inserted one). In total, $\frac{n(n+1)}{2} - 3$ insertions are carried out, being n of these complete sequences at the last iteration. Therefore, the computational complexity of the NEH is $\mathcal{O}(n^3m)$. A closer look shows that when inserting the k -th job in position, let us say, j , all $C_{i,h}$, $h = \{j - 1, j - 2, \dots, 1\}$ were already calculated in the previous insertion and there is no need to recalculate. Taillard [46] exploited this idea by which all insertions in a given step can be calculated in $\mathcal{O}(nm)$ effectively reducing the complexity of the NEH to $\mathcal{O}(n^2m)$. This faster variant is referred to as NEHT or NEH with Taillard's accelerations.

The NEH was computationally tested against 25 other heuristics, including other more recent and complex heuristics in Ruiz and Maroto [47]. The experiments demonstrated that the NEH is superior to all other tested methods and at the same time much faster. This was supported by statistical analyses. In the results, NEH showed an average percentage deviation from the best solutions known in the 120 instances of Taillard [48] of just 3.33%. Although this is an average result, NEH rarely goes beyond 7%. Furthermore, NEH is actively used as a seed sequence for metaheuristic techniques.

The NEHT was also evaluated by Rad et al. [49] with surprising results. For example, in the 10 instances of the largest size (500×20) from the benchmark of Taillard [48], NEHT managed to give an average percentage deviation from the best known solutions (these solutions have been proved to be “almost optimal” with very small gaps between the highest known lower bound and lowest known upper

bound or best solution known) of just 2.24%. Furthermore, this 2.24% deviation from a very close to optimum solution was achieved in an average of just 0.0773 s. Extensions of the NEH presented in the same paper of Rad et al. [49] resulted in much better performance, reaching an average percentage deviation from best known solutions of just 1.4% in CPU times under 2 s.

With these results, it is easy to see why the NEH is such an important and widely studied heuristic. Studies about the NEH have been numerous over the years. The papers of [49–57] and more recently Fernandez-Viagas and Framinan [58] are some examples. Furthermore, the NEH does not rely on specific problem knowledge, and it is relatively easy to extend it and apply it to different scheduling problems. Such extensions are numerous in the scheduling literature, mainly over flow shop problems but also in hybrid settings and other problems.

A Brief Introduction to the Shifting Bottleneck Heuristic for the Job Shop Problem

The shifting bottleneck heuristic or SBH for short was initially proposed by Adams et al. [59] for the job shop problem and makespan minimization ($F//C_{\max}$). Many improvements on the initial SBH for the same problem were presented over the following years. Additionally, SBH has been extended by many authors to other objectives and problems. It is a very interesting method because it mimics the reality of a production floor. In real factories one production stage or machine is usually the bottleneck of the whole plant, i.e., productivity or the measured objective is hard to improve unless the scheduling or capacity of this stage is improved. SBH tries to optimize the schedule of the bottleneck machine first as the contribution of this machine to the final makespan is large. Since in a job shop there are m machines and jobs are made of many tasks with precedence relationships among them, scheduling only the bottleneck machine is not straightforward. The procedure can be simply described as follows:

1. Determine the bottleneck machine.
2. Schedule all previous tasks of the n jobs on the other machines to obtain the release dates (r_j) of the jobs on the bottleneck machine.
3. Schedule all subsequent tasks of the n jobs on the other machines to obtain the due dates (d_j) of the jobs on the bottleneck machine.
4. Schedule the bottleneck machine as a single machine problem with release dates and due dates, minimizing the maximum lateness ($1/r_j, d_j/\max L_j$).
5. Repeat all previous steps for the remaining $m - 1$ machines until all machines have a sequence.

Note that the single machine problem is solved m times. The $1/r_j, d_j/\max L_j$ is in itself a hard problem. However, efficient methods do exist for its solution. In particular, the original SBH heuristic of [59] uses the algorithm of [60]. Additional details about the SBH procedure, including examples, are given in [1] and in [3].

The performance of the SBH is outstanding, especially if one considers that the job shop problem is incredibly hard to solve. Back at the time the authors were able to solve problems of up to 50 jobs and 10 machines to optimality in some cases. As a result, the literature on the SBH is incredibly large. More details about this heuristic and many other related bottleneck decomposition methods are given in the book of [61].

Metaheuristics

While heuristics are usually devised for specific problems with a given objective function, they are, in principle, more generic. This distinction is far from clear as many heuristics are indeed general as well, like the aforementioned NEH. In any case, metaheuristics can be seen as general algorithmic templates that exist without the particular problem or application. Additionally, metaheuristics are commonly more demanding and CPU intensive than regular heuristics. The desired effect of this additional computation is a better solution quality. Other common characteristics of metaheuristics are mechanisms to escape local optimality and to reduce the effect of the myopic nature of heuristic approaches by diversification mechanisms. The field of metaheuristics is interdisciplinary and terribly prolific with literally thousands of papers being published every year in many different fields, not to mention the large number of monographs dedicated to them. Scheduling being an important problem, the application of metaheuristics to scheduling is frequent. A good starting point are the books of [30, 62].

Metaheuristics are roughly grouped into families or classes. Each family stems from a basic metaheuristic template that, with relatively few modifications, can be applied to a specific scheduling problem, and reasonable results might be obtained. To obtain very high-quality solutions, more specific operators and local search schemes are usually needed. This also results in slower methods. The main categories are simulated annealing, tabu search, genetic algorithms, ant colony optimization, scatter search, variable neighborhood search, GRASP and iterated local search, etc. In the following section, the main constructs inside most metaheuristic algorithms are described, to be followed by a brief description of a successful metaheuristic that is at the same time simple: iterated greedy.

Main Concepts of Metaheuristics in Scheduling

Most metaheuristic classes require at least the instantiation of some of the following elements:

- Representation of the solution. At the lowest level, solutions contain variables and these might be continuous, discrete, binary, etc. A representation is just how the variables are represented and arranged. In scheduling problems that require permutations and an array or list of integers, each one representing a job is

an indirect representation of the final solution. Such a solution is “scheduled” (i.e., starting and finish times for each job and/or task are calculated) in order to calculate the desired objective function. For parallel machine problems where only the assignment is relevant (e.g., the $P//C_{\max}$), each job might have an integer indicating the machine to which it is assigned to. The representation is very important for the final performance of the metaheuristic.

- Initial solution or population generation. Most metaheuristics require an initial solution to start the process. Population-based metaheuristics require a set of solutions. The easiest procedure is to randomly generate these initial solutions. Of course, this results in initial solutions of mediocre quality. The most commonly employed alternative is to use high-performance heuristics to generate initial schedules.
- Intensification and diversification mechanisms. Initial solutions are worked on through local search and/or other procedures like selection and crossover in genetic algorithms. The objective is to find new and better solutions. There are many procedures and techniques but all of these fall more or less into what is referred to as intensification. To avoid stagnation in the search process, most metaheuristics include diversification mechanisms by which existing solutions are disrupted or altered (or new solutions are generated) in order to allow the intensification procedures to work over different parts of the solution and search space.
- Acceptance criterion. New solutions generated through the intensification and diversification processes have to be examined in order to decide if they replace current incumbent solutions or not. The simplest scenario is when new solutions are accepted if and only if they improve on the current or best objective function value found so far during the search. This criterion leads to premature convergence and stagnation, and most metaheuristic templates consider more elaborated alternatives where worse solutions might be accepted, often temporarily.
- Termination criterion. Metaheuristics iterate continuously unless a user-given termination criterion is met. For most metaheuristics, optimality is not guaranteed. Furthermore, most applications of metaheuristics do not have an indication of when an optimal solution has been found in order to stop the search. Therefore, common termination criteria include a number of iterations or a consecutive number of iterations without improvements (or small improvements), elapsed CPU time, etc.

The previous operators are only a very small list where not even all of them might be needed to construct a simple method. Furthermore, elaborated metaheuristics might have many operators and procedures. Actually, in the last decade, many natural or man-made processes have inspired a real tsunami of metaheuristic methods. As a result there are algorithms that are referred to as particle swarm, imperialist competitive, honey bees, fruit flies, and even more bizarre methods like intelligent water drops or galaxy optimization. Currently, such “novel” methods are

being criticized. The papers of [63, 64] are two examples. In this chapter these criticisms are supported and very successful, but at the same time really simple metaheuristic is described.

Iterated Greedy as an Example of a Simple- and High-Performing Method

The first application of the concepts of iterated greedy (IG) to a scheduling problem was presented by [65]. The authors dealt with the permutation flow shop problem with makespan criterion ($F|prmu/C_{\max}$). Being a metaheuristic, IG starts from an initial heuristic solution. Then three phases are iteratively applied until a termination criterion is met. The first phase is a partial destruction of the current solution. Some jobs are randomly removed from the permutation. The second phase is the reconstruction of the (now) incomplete solution. All removed jobs are inserted, one by one, into all positions in the partial solution using the NEH heuristic. Each job is placed into the position resulting in the best objective function value. An optional step is a local search each time a complete solution is obtained. This local search is carried out in the insertion neighborhood until local optimality. The third last step is an acceptance criterion. A detailed pseudoalgorithm listing is given in Fig. 3.

As one can see, IG carries out intensification with the reconstruction and local search operators and diversification with the destruction. The acceptance criterion is a simplification of simulated annealing. With a constant temperature factor, worse solutions are accepted with a probability that decreases with the quality of the new solution.

The results of the IG method for the permutation flow shop problem are state-of-the-art even today. According to the results of [65], IG was able to reduce the average relative percentage deviation from the best known solutions in the 120 instances of [48] to just 0.44% in little under a minute on average. It was computationally and statistically demonstrated that IG produced better results than competing methods. Later, [66] were able to further reduce this deviation to 0.22% using a simple island model parallel version of the IG.

IG has been applied to different scheduling problems with good results. Ruiz and Stützle [67] extended the previous results with the consideration of sequence-dependent setup times. IG for no wait and blocking flow shops were studied by [68, 69], respectively. IG showed excellent performance in no-idle and mixed no-idle flow shops in [70] and [71]. Non-permutation flow shops were approached by [72]. Distributed scheduling problems have been solved with IG methods by [73, 74]. IG also gives good results in other objectives aside from makespan like tardiness in [75] or total flow time in [76], to name just a few. Multi-objective extensions of IG have proven effective for Pareto flow shops without and with setups in [77] and in [78], respectively. Parallel machine problems were successfully solved with IG methods by [79]. Finally, hybrid flow shops were studied by [80] and complex

procedure *Iterated_Greedy*

```

 $\pi := \text{NEH\_Heuristic};$ 

 $\pi := \text{Insertion\_LocalSearch}(\pi);$ 

 $\pi_b := \pi;$ 

while (termination criterion not satisfied) do

     $\pi' := \pi;$ 

    for  $i := 1$  to destruct do % Destruction phase

         $\pi' :=$  randomly extract a job of  $\pi'$  and insert it into  $\pi'_R$ ;

    for  $i := 1$  to destruct do % Reconstruction phase

         $\pi' :=$  best permutation after inserting  $\pi_R(i)$  into all possible positions of  $\pi'$ ;

     $\pi'' := \text{Insertion\_LocalSearch}(\pi');$  % Local Search

    if  $C_{\max}(\pi'') < C_{\max}(\pi)$  then  $\pi := \pi'';$  % Acceptance criterion

        if  $C_{\max}(\pi) < C_{\max}(\pi_b)$  then  $\pi_b := \pi;$ 

        elseif  $\left( \text{random} \leq e^{-\frac{(C_{\max}(\pi'') - C_{\max}(\pi))}{\text{Temperature}}} \right)$  then  $\pi := \pi'';$ 

    endif

endwhile

return  $\pi_b$ 

end

```

Fig. 3 Iterated greedy (IG) algorithm. (Adapted from [65]). *random* is a random number distributed uniformly in the range [0, 1]. *Temperature* and *destruct* are the two parameters of IG

realistic hybrid flow shops by [81–83]. There are many other applications of the IG methodology to other scheduling problems and also to several other combinatorial problems as well.

What is noteworthy about all previous studies is that IG algorithms are shown to produce state-of-the-art results that on many occasions are competitive or better than those produced by other much more complex metaheuristics.

There are highly successful applications of different metaheuristics for practically every formulated scheduling problem. Many are listed in [3] and the references therein.

Conclusions

In this chapter the scheduling field has been introduced, pointing out its relevance, difficulty, and hardness. Scheduling problems are varied, hard, and difficult to solve optimally. However, in many settings, obtaining approximated solutions of sufficient quality is within reach if modern heuristic methodologies are employed. The notation of scheduling problems has been introduced. A succinct explanation of simple dispatching rules is provided. An example that shows the lack of correlation between otherwise seemingly correlated objectives has been presented which again highlights the fact that scheduling problems are neither easy nor intuitive. Furthermore, in a briefly summarized application, it has been shown how a very simple dispatching rule can result in almost optimal solutions for real problems of a large size. Even though elaborated and combined dispatching rules are powerful, advanced heuristic methods are many times better. The NEH method for the flow shop and the SBH for the job shop are the clearest examples. High-performance and close-to-optimal solutions are perfectly possible in negligible CPU times. Finally, in an ever-growing field, metaheuristics propose themselves as modern heralds of flexibility, robustness, and finest solution quality. A summary of the iterated greedy method has been presented which apart from the previous qualities manages state-of-the-art results without unnecessary complexities in its design.

As for the future of scheduling heuristics, it is argued for even simpler and more robust approaches, capable of solving hard and varied scheduling problems with little to no instantiation and adaptation. Such frameworks and methodologies could prove invaluable in closing the gap between the theory and practice of scheduling.

Cross-References

- [Iterated Greedy](#)
- [Iterated Local Search](#)

Acknowledgments The author is partially supported by the Spanish Ministry of Economy and Competitiveness, under the project “SCHEYARD – Optimization of Scheduling Problems in Container Yards” (No. DPI2015-65895-R) financed by FEDER funds.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Pinedo ML (2012) Scheduling: theory, algorithms and systems, 4th edn. Springer, New York
2. Pinedo ML (2009) Planning and scheduling in manufacturing and services, 2nd edn. Springer, New York
3. Framinan JM, Leisten R, Ruiz R (2014) Manufacturing scheduling systems. An integrated view on models, methods and tools. Springer, New York

4. Błażewicz J, Ecker KH, Pesch E, Schmidt G, J W (2001) Scheduling computer and manufacturing processes, 2nd edn. Springer, Berlin
5. Grosche T (2009) Computational intelligence in integrated airline scheduling. Studies in computational intelligence, vol 173. Springer, New York
6. Randolph H (ed) (2012) Handbook of healthcare system scheduling. International series in operations research & management science, vol 168. Springer, New York
7. Briskorn D (2008) Sports leagues scheduling. Models, combinatorial properties, and optimization algorithms. Lecture notes in economics and mathematical systems, vol 603. Springer, Berlin/Heidelberg
8. Graves SC (1981) A review of production scheduling. *Oper Res* 29(4):646–675
9. Ledbetter WN, Cox JF (1977) Operations research in production management: an investigation of past and present utilisation. *Prod Invent Manag* 18(3):84–91
10. Ford FN, Bradbard DA, Ledbetter WN, Cox JF (1987) Use of operations research in production management. *Prod Invent Manag* 28(3):59–62
11. McKay KN, Safayeni FR, Buzacott JA (1988) Job-shop scheduling theory: what is relevant? *Interfaces* 4(18):84–90
12. Olhager J, Rapp B (1995) Operations research techniques in manufacturing planning and control systems. *Int Trans Oper Res* 2(1):29–43
13. Dudek RA, Panwalkar SS, Smith ML (1992) The lessons of flowshop scheduling research. *Oper Res* 40(1):7–13
14. McCarthy BL, Liu J (1993) Addressing the gap in scheduling research: a review of optimization and heuristic methods in production scheduling. *Int J Prod Res* 31(1):59–79
15. McKay KN, Wiers VCS (1999) Unifying the theory and practice of production scheduling. *J Manuf Syst* 18(4):241–255
16. McKay KN, Pinedo ML, Webster S (2002) Practice-focused research issues for scheduling systems. *Prod Oper Manag* 11(2):249–258
17. McKay KN, Wiers VCS (2006) The human factor in planning and scheduling, chap 2. In: Herrmann JW (ed) Handbook of production scheduling. International series in operations research & management science, vol 89. Springer, New York, pp 23–57
18. Reisman A, Kumar A, Motwani J (1997) Flowshop scheduling/sequencing research: a statistical review of the literature, 1952–1994. *IEEE Trans Eng Manag* 44(3):316–329
19. Ribas I, Leisten R, Framinan JM (2010) Review and classification of hybrid flow shop scheduling problems from a production system and a solutions procedure perspective. *Comput Oper Res* 37(8):1439–1454
20. Ruiz R, Vázquez-Rodríguez JA (2010) The hybrid flowshop scheduling problem. *Eur J Oper Res* 205(1):1–18
21. Hopp WJ, Spearman ML (2011) Factory physics, 3rd edn. Waveland Press Inc., Long Grove
22. McKay KN, Wiers VCS (2004) Practical production control. A survival guide for planners and schedulers. J. Ross Publishing Inc., Boca Raton
23. Brucker P (2007) Scheduling algorithms, 5th edn. Springer, New York
24. Laporte G (2009) Fifty years of vehicle routing. *Transp Sci* 43(4):408–416
25. Baldacci R, Mingozzi A, Roberti R (2011) New route relaxation and pricing strategies for the vehicle routing problem. *Oper Res* 59(5):1269–1283
26. Brucker P, Jurisch B, Sievers B (1994) A branch and bound algorithm for the job-shop scheduling problem. *Discret Appl Math* 49(1–3):107–127
27. Framinan JM, Ruiz R (2010) Architecture of manufacturing scheduling systems: literature review and an integrated proposal. *Eur J Oper Res* 205(2):237–246
28. Framinan JM, Ruiz R (2012) Guidelines for the deployment and implementation of manufacturing scheduling systems. *Int J Prod Res* 50(7):1799–1812
29. Morton TE, Pentico DW (1993) Heuristic scheduling systems with applications to production systems and project management. Wiley series in engineering & technology management. Wiley, Hoboken
30. Hoos HH, Stützle T (2005) Stochastic local search: foundations and applications. Morgan Kaufmann, San Francisco

31. Cai X, Wu X, Zhou X (2014) Optimal stochastic scheduling. International series in operations research & management science, vol 207. Springer, New York
32. Sarin SC, Nagarajan B, Liao L (2014) Stochastic scheduling. Expectation-variance analysis of a schedule. Cambridge University Press, Cambridge
33. Rinnooy Kan AHG (1976) Machine scheduling problems: classification, complexity and computations. Martinus Nijhoff, The Hague
34. Vignier A, Billaut JC, Proust C (1999) Les problèmes d'ordonnancement de type flow-shop hybride: État de l'art. *RAIRO Recherche opérationnelle* 33(2):117–183. (in French)
35. T'Kindt V, Billaut JC (2006) Multicriteria scheduling: theory, models and algorithms, 2nd edn. Springer, New York
36. Blackstone JH Jr, Phillips DT, Hogg GL (1982) A state-of-the-art survey of dispatching rules for manufacturing job shop operations. *Int J Prod Res* 20(1):27–45
37. Haupt R (1989) A survey of priority rule-based scheduling. *OR Spectr* 11(1):3–16
38. Jayamohan MS, Rajendran C (2000) New dispatching rules for shop scheduling: a step forward. *Int J Prod Res* 38(3):563–586
39. Panwalkar SS, Iskander W (1977) A survey of scheduling rules. *Oper Res* 25(1):45–61
40. Lenstra JK, Rinnooy Kan AHG, Brucker P (1977) Complexity of machine scheduling problems. *Ann Discret Math* 1:343–362
41. Graham RL (1969) Bounds on multiprocessing timing anomalies. *SIAM J Appl Math* 17(2):416–429
42. Vepsäläinen APJ, Morton TE (1987) Priority rules and lead time estimation for job shop scheduling with weighted tardiness costs. *Manag Sci* 33(8):1036–1047
43. Lee YH, Bhaskaran K, Pinedo ML (1997) A heuristic to minimize the total weighted tardiness with sequence-dependent setups. *IIE Trans* 29(1):45–52
44. Nawaz M, Ensore EE Jr, Ham I (1983) A heuristic algorithm for the m -machine, n -job flow-shop sequencing problem. *OMEGA Int J Manag Sci* 11(1):91–95
45. Turner S, Booth D (1987) Comparison of heuristics for flow shop sequencing. *OMEGA Int J Manag Sci* 15(1):75–78
46. Taillard E (1990) Some efficient heuristic methods for the flow shop sequencing problem. *Eur J Oper Res* 47(1):67–74
47. Ruiz R, Maroto C (2005) A comprehensive review and evaluation of permutation flowshop heuristics. *Eur J Oper Res* 165(2):479–494
48. Taillard E (1993) Benchmarks for basic scheduling problems. *Eur J Oper Res* 64(2):278–285
49. Rad SF, Ruiz R, Boroojerdian N (2009) New high performing heuristics for minimizing makespan in permutation flowshops. *OMEGA Int J Manag Sci* 37(2):331–345
50. Chakraborty UK, Laha D (2007) An improved heuristic for permutation flowshop scheduling. *Int J Inf Commun Technol* 1(1):89–97
51. Dong XY, Huang HK, Chen P (2006) A more effective constructive algorithm for permutation flowshop problem. In: Intelligent data engineering and automated learning (IDEAL 2006), Lecture notes in computer science, vol 4224. Springer, Berlin/New York (Beijing Jiaotong Univ, Sch Comp & IT, Beijing 100044, Peoples R China), pp 25–32
52. Dong XY, Huang HK, Chen P (2008) An improved NEH-based heuristic for the permutation flowshop problem. *Comput Oper Res* 35(12):3962–3968
53. Framinan JM, Leisten R, Rajendran C (2003) Different initial sequences for the heuristic of Nawaz, Ensore and Ham to minimize makespan, idletime or flowtime in the static permutation flowshop sequencing problem. *Int J Prod Res* 41(1):121–148
54. Jin F, Song SJ, Wu C (2007) An improved version of the NEH algorithm and its application to large-scale flow-shop scheduling problems. *IIE Trans* 39(2):229–234
55. Kalczynski PJ, Kamburowski J (2007) On the NEH heuristic for minimizing the makespan in permutation flow shops. *OMEGA Int J Manag Sci* 35(1):53–60
56. Kalczynski PJ, Kamburowski J (2008) An improved NEH heuristic to minimize makespan in permutation flow shops. *Comput Oper Res* 35(9):3001–3008
57. Kalczynski PJ, Kamburowski J (2009) An empirical analysis of the optimality rate of flow shop heuristics. *Eur J Oper Res* 198(1):93–101

58. Fernandez-Viagas V, Framinan JM (2014) On insertion tie-breaking rules in heuristics for the permutation flowshop scheduling problem. *Comput Oper Res* 45:60–67
59. Adams J, Balas E, Zawack D (1988) The shifting bottleneck procedure for job shop scheduling. *Manag Sci* 34(3):391–401
60. Carlier J (1982) The one-machine sequencing problem. *Eur J Oper Res* 11(1):42–47
61. Ovacik IM, Uzsoy R (1997) Decomposition methods for complex factory scheduling problems. Springer, New York
62. El-Ghazali T (2009) Metaheuristics: from design to implementation. Wiley, New York
63. Sörensen K (2015) Metaheuristics—the metaphor exposed. *Int Trans Oper Res* 22(1):3–18
64. Weyland D (2010) A rigorous analysis of the harmony search algorithm: how the research community can be misled by a “novel” methodology. *Int J Appl Metaheuristic Comput* 1(2):50–60
65. Ruiz R, Stützle T (2007) A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *Eur J Oper Res* 177(3):2033–2049
66. Vallada E, Ruiz R (2009) Cooperative metaheuristics for the permutation flowshop scheduling problem. *Eur J Oper Res* 193(2):365–376
67. Ruiz R, Stützle T (2008) An iterated greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives. *Eur J Oper Res* 187(3):1143–1159
68. Pan QK, Wang L, Zhao BH (2008) An improved iterated Greedy algorithm for the no-wait flow shop scheduling problem with makespan criterion. *Int J Adv Manuf Technol* 38(7–8):778–786
69. Ribas I, Companys R, Tort-Martorell X (2011) An iterated greedy algorithm for the flowshop scheduling problem with blocking. *OMEGA Int J Manag Sci* 39(3):293–301
70. Ruiz R, Vallada E, Fernández-Martínez C (2009) Scheduling in flowshops with no-idle machines. In: Chakraborty UK (ed) *Computational intelligence in flow shop and job shop scheduling*. Studies in computational intelligence, vol 230. Springer, Berlin, pp 21–51
71. Pan QK, Ruiz R (2014) An effective iterated Greedy algorithm for the mixed no-idle flowshop scheduling problem. *OMEGA Int J Manag Sci* 44(1):41–50
72. Ying KC (2008) Solving non-permutation flowshop scheduling problems by an effective iterated Greedy heuristic. *Int J Adv Manuf Technol* 38(3–4):348–354
73. Fernandez-Viagas V, Framinan JM (2015) A bounded-search iterated greedy algorithm for the distributed permutation flowshop scheduling problem. *Int J Prod Res* 53(4):1111–1123
74. Lin SW, Ying KC, Huang CY (2013) Minimising makespan in distributed permutation flowshops using a modified iterated greedy algorithm. *Int J Prod Res* 51(16):5029–5038
75. Framinan JM, Leisten R (2008) Total tardiness minimization in permutation flow shops: a simple approach based on a variable greedy algorithm. *Int J Prod Res* 46(22):6479–6498
76. Pan QK, Ruiz R (2012) Local search methods for the flowshop scheduling problem with flowtime minimization. *Eur J Oper Res* 222(1):31–43
77. Minella G, Ruiz R, Ciavotta M (2011) Restarted iterated pareto greedy algorithm for multi-objective flowshop scheduling problems. *Comput Oper Res* 38(11):1521–1533
78. Ciavotta M, Minella G, Ruiz R (2013) Multi-objective sequence dependent setup times flowshop scheduling: a new algorithm and a comprehensive study. *Eur J Oper Res* 227(2):301–313
79. Fanjul-Peyro L, Ruiz R (2010) Iterated greedy local search methods for unrelated parallel machine scheduling. *Eur J Oper Res* 207(1):55–69
80. Ying KC (2008) An iterated Greedy heuristic for multistage hybrid flowshop scheduling problems with multiprocessor tasks. *IEEE Trans Evol Comput* 60(6):810–817
81. Urlings T, Ruiz R (2007) Local search in complex scheduling problems. In: Stützle T, Birattari M, Hoos HH (eds) *Engineering stochastic local search algorithms. Designing, implementing and analyzing effective heuristics*, Lecture notes in computer science, vol 4638. Springer, Brussels, pp 202–206
82. Urlings T, Ruiz R, Sivrikaya-Şerifoğlu F (2010) Genetic algorithms for complex hybrid flexible flow line problems. *Int J Met* 1(1):30–54
83. Urlings T, Ruiz R, Stützle T (2010) Shifting representation search for hybrid flexible flowline problems. *Eur J Oper Res* 207(2):1086–1095